

Documentation technique

Créer et administrer une machine virtuelle sous Proxmox VE 9

Objet : procédure de référence pour provisionner une VM, la piloter en ligne de commande, ajuster ses ressources et gérer ses snapshots. **Public** : technicien ou administrateur système intervenant sur un hyperviseur Proxmox VE 9. **Rédigée le** : 22/09/2026 · **Testée sur** : Proxmox VE 9.2.2, nœud `pve-haitem`

Sommaire

1. [Prérequis et accès](#)
 2. [Préparer une image ISO](#)
 3. [Créer une machine virtuelle](#)
 4. [Démarrer, arrêter, supprimer](#)
 5. [Piloter en ligne de commande](#)
 6. [Modifier les ressources](#)
 7. [Gérer les snapshots](#)
 8. [Dépannage](#)
 9. [Mémento des commandes](#)
-

1. Prérequis et accès

1.1 Interface web

```
https://<adresse_du_serveur>:8006
```

Le certificat est auto-signé : le navigateur affiche un avertissement, qu'il faut accepter (*Avancé* → *Continuer vers le site*). Authentification avec le realm `Linux PAM standard authentication` et le compte `root`.

1.2 Accès à un terminal

Trois possibilités, par ordre de préférence :

Méthode	Accès	Remarque
SSH	<code>ssh root@<adresse></code>	À privilégier : terminal complet, transfert de fichiers
Shell intégré	<code><node></code> → <code>Shell</code>	Solution de repli si le SSH est indisponible — terminal root équivalent
Console d'une VM	<code><VM></code> → <code>Console</code>	Accès à l'intérieur de la VM, pas à l'hyperviseur

Le Shell intégré est très pratique en formation : il permet de garder l'interface graphique et le terminal côte à côte pour comparer les deux méthodes.

1.3 Activer le copier-coller dans le Shell intégré

Le viewer console se règle dans `Datacenter` → `Options` → `Console Viewer` :

Viewer	Copier-coller	Usage
xterm.js	☒ <code>Ctrl+Shift+C</code> / <code>Ctrl+Shift+V</code>	shell texte de l'hyperviseur
noVNC	☐ flux d'images	console graphique d'une VM

Si les collages arrivent entourés de caractères parasites (`^[[200~`), désactiver le *bracketed paste* de façon permanente :

```
echo 'set enable-bracketed-paste off' >> ~/.inputrc
```

1.4 Vérifier l'environnement avant toute intervention

```
pveversion          # version de Proxmox
hostname            # nom du nœud
qm list             # VM existantes et VMID déjà pris
pvesm status        # stockages disponibles et espace libre
```

2. Préparer une image ISO

Une VM se crée à **partir d'une image ISO**, pas d'un modèle. (Les modèles — *templates* — concernent les conteneurs LXC, ou une VM convertie en template.)

Méthode graphique

<node> → local → ISO Images → Download from URL, puis coller l'URL de l'image.

Méthode en ligne de commande

```
cd /var/lib/vz/template/iso
wget <url_de_l_iso>
ls -lh
```

Le chemin `/var/lib/vz/template/iso` correspond au stockage `local`. Une image déposée là apparaît immédiatement dans l'interface.

3. Créer une machine virtuelle

Bouton **Create VM**, en haut à droite. L'assistant se parcourt onglet par onglet avec **Next**.

Onglet	Paramètre	Choix et justification
General	Node	nœud d'accueil
	VM ID	identifiant numérique unique — c'est lui qui sert dans toutes les commandes
	Name	étiquette d'affichage, sans rôle technique
OS	ISO image	image d'installation
	Type / Version	conditionne le matériel virtuel généré — voir l'encadré ci-dessous
System	BIOS	SeaBIOS (classique) ou OVMF (UEFI) (moderne, requis pour Secure Boot)

Onglet	Paramètre	Choix et justification
	Machine	<code>i440fx</code> (compatibilité) ou <code>q35</code> (moderne, obligatoire avec UEFI)
	Qemu Agent	à cocher — permet l'arrêt propre et la remontée d'informations depuis l'invité
Disks	Bus	<code>VirtIO SCSI</code> = performances maximales, mais exige un pilote dans l'invité . <code>SATA</code> ou <code>IDE</code> = universellement reconnus
	Storage	<code>local-lvm</code> pour les disques de VM
	Disk size	dimensionner selon l'OS installé
CPU	Sockets × Cores	vCPU total = sockets × cores
	Type	<code>host</code> expose le CPU réel (meilleures performances, empêche la migration vers un hôte différent)
Memory	Memory (MiB)	valeur en mébiotets : 4096 = 4 Gio
Network	Bridge	<code>vmbro0</code> par défaut
	Model	<code>VirtIO</code> = performant mais nécessite un pilote ; <code>Intel E1000</code> = reconnu nativement par tous les OS

Le champ *OS Type* n'est pas une simple étiquette. Sélectionner un profil Windows 11 déclenche la création automatique d'un disque `efidisk0` (variables du firmware UEFI) et d'un disque `tpmstate0` (module TPM 2.0 virtuel), tous deux de 4 Mo, et impose la machine `q35`. Proxmox adapte le matériel virtuel aux exigences du système déclaré.

Bonne pratique : décocher *Start after created* pour contrôler soi-même le premier démarrage et observer le changement d'état.

L'onglet **Confirm** récapitule l'intégralité des paramètres : c'est l'écran à conserver comme trace de la configuration initiale.

4. Démarrer, arrêter, supprimer

4.1 Démarrer

Sélectionner la VM → **Start**, puis **Console** pour suivre l'amorçage.

Sur une VM neuve, l'ordre de démarrage (`boot: order=...`) tente le disque en premier ; celui-ci étant vide, le firmware bascule sur le lecteur CD et lance l'installateur.

4.2 Arrêter — deux mécanismes à ne pas confondre

Bouton	Mécanisme	Quand l'utiliser
Shutdown	Signal ACPI envoyé au système invité, qui déclenche sa propre extinction	Par défaut — arrêt propre, sans risque pour les données
Stop	Arrêt du processus QEMU sur l'hyperviseur	Uniquement si l'invité ne répond plus

Shutdown reste **sans effet** si aucun système d'exploitation n'écoute le signal : VM sans OS installé, système figé, ou invité dépourvu du service ACPI. Ce n'est pas une panne de l'hyperviseur. **Stop** équivaut à une coupure de courant : il fonctionne toujours, mais expose à une corruption de données si l'invité écrivait sur le disque.

4.3 Supprimer

`<VM>` → **More** → **Remove**. Proxmox impose une double confirmation : **retaper le VMID** dans un champ de saisie, et un avertissement en surbrillance « **Referenced disks will always be destroyed** ».

Cet avertissement est à lire attentivement : **les disques référencés dans la configuration sont détruits dans tous les cas**, indépendamment des cases à cocher. Ces dernières ne couvrent que ce qui se situe hors configuration :

Option	Portée
Purge from job configurations	Retire la VM des tâches de sauvegarde et de réplication — évite des jobs qui échoueraient ensuite sur une VM inexistante
Destroy unreferenced disks owned by guest	Détruit les disques <i>orphelins</i> de la VM : un disque détaché de la configuration mais jamais effacé du stockage

Les cocher toutes les deux est la bonne pratique : sinon des résidus subsistent sur le stockage et des tâches planifiées pointent dans le vide.

⚠ **Opération irréversible.** Avant de supprimer : vérifier qu'aucune tâche n'est en cours sur cette VM, et qu'une sauvegarde récente existe si les données doivent être conservées.

En ligne de commande, l'équivalent est `qm destroy <vmid> --purge 1 --destroy-unreferenced-disks 1`.

5. Piloter en ligne de commande

5.1 Les outils

Commande	Périmètre
<code>qm</code>	machines virtuelles (KVM/QEMU)
<code>pct</code>	conteneurs LXC
<code>pvesm</code>	stockages
<code>pvecm</code>	cluster
<code>pvesh</code>	accès direct à l'API REST
<code>vzdump</code>	sauvegardes

Règle absolue : on désigne une VM par son VMID numérique, jamais par son nom. `qm start 101` fonctionne, `qm start TestVM1` échoue.

5.2 Opérations courantes

<code>qm list</code>	# lister les VM, leur statut et leurs ressources
<code>qm start <vmid></code>	# démarrer
<code>qm shutdown <vmid></code>	# arrêt propre (ACPI) – à privilégier
<code>qm stop <vmid></code>	# arrêt forcé
<code>qm reset <vmid></code>	# redémarrage brutal

```
qm status <vmid> # statut synthétique
```

En environnement Unix, l'absence de sortie signale le succès : `qm stop 101` n'affiche rien quand tout se passe bien.

5.3 Obtenir le détail d'une VM

Deux commandes qui répondent à deux questions différentes :

```
qm config <vmid> # ce que la VM EST (statique)
qm status <vmid> --verbose # ce que la VM FAIT (dynamique)
```

Commande	Source des données
<code>qm config</code>	fichier <code>/etc/pve/qemu-server/<vmid>.conf</code>
<code>qm status --verbose</code>	processus QEMU en cours d'exécution

La sortie de `--verbose` compte environ 200 lignes triées alphabétiquement. Pour n'afficher que les champs de premier niveau :

```
qm status <vmid> --verbose | grep ^[a-z]
```

Champs les plus utiles : `status`, `cpus`, `mem` / `maxmem`, `maxdisk`, `pid`, `uptime`, `running-qemu`.

Lire l'écart `mem` / `maxmem`. Une VM à qui l'on a alloué 2 Gio n'en consomme souvent que quelques centaines de Mio sur l'hôte : c'est l'**allocation paresseuse**, la mémoire n'étant réservée qu'au fur et à mesure des accès. C'est ce qui autorise le surprovisionnement. Le *ballooning*, qui permet à l'hôte de **reprendre** de la mémoire, exige en revanche un pilote installé dans le système invité.

5.4 Architecture : pourquoi les deux méthodes sont équivalentes



L'interface web est un client JavaScript de l'API REST ; `qm` en est un client en ligne de commande. Les deux écrivent dans le **même fichier de configuration**. Une modification par clic est immédiatement visible dans `qm config`, et réciproquement.

Conséquence pratique : les erreurs de `qm` sont des **codes HTTP**. Un `400 too many arguments` signale une requête mal formée, pas un dysfonctionnement du serveur.

Quand choisir quoi : l'interface pour une intervention ponctuelle et exploratoire ; la ligne de commande dès qu'il faut répéter, scripter ou traiter plusieurs machines d'un coup.

6. Modifier les ressources

6.1 Règle préalable

Arrêter la VM avant toute modification matérielle. Les changements de mémoire et de CPU ne prennent effet qu'au démarrage suivant (hors configuration explicite de hotplug).

6.2 En ligne de commande

```
qm set <vmid> --memory 4096      # RAM en MiB
qm set <vmid> --cores 2           # cœurs par socket
qm set <vmid> --sockets 1         # nombre de sockets
```

Paramètre	Signification
--memory	mémoire en mébiotets (4096 = 4 Gio)
--sockets	processeurs physiques simulés
--cores	cœurs par socket
vCPU total	<code>sockets × cores</code>

6.3 Dans l'interface

<VM> → **Hardware** → double-clic sur la ligne à modifier.

La boîte de dialogue **Memory** révèle trois options complémentaires dès qu'on modifie la valeur :

Option	Effet
Minimum memory	plancher sous lequel l'hyperviseur ne descendra pas. Égal au maximum ⇒ allocation fixe

Option	Effet
Ballooning Device	autorise l'hôte à récupérer de la mémoire sous pression
Allow KSM	<i>Kernel Same-page Merging</i> : fusion des pages mémoire identiques entre VM, très efficace quand plusieurs machines exécutent le même OS

6.4 Vérifier

```
qm config <vmid> | grep -E '^(memory|cores|sockets)'
```

7. Gérer les snapshots

7.1 Principe

Un snapshot fige l'état de la VM à un instant T. Les écritures suivantes sont enregistrées séparément, ce qui permet de revenir en arrière à tout moment.

Sur un stockage `local-lvm`, Proxmox crée **un volume logique LVM-thin par disque virtuel** — disque système, disque EFI, état du TPM. Le mécanisme est de la **copie sur écriture** : rien n'est copié à la création, d'où son instantanéité.

7.2 Commandes

Action	Commande	Équivalent interface
Créer	<code>qm snapshot <vmid> <nom> [--description "..."]</code>	Snapshots → Take Snapshot
Lister	<code>qm listsnapshot <vmid></code>	onglet Snapshots
Restaurer	<code>qm rollback <vmid> <nom></code>	Rollback
Supprimer	<code>qm delsnapshot <vmid> <nom></code>	Remove

7.3 Capturer aussi la mémoire

Sur une VM **en fonctionnement** :

```
qm snapshot <vmid> <nom> --vmstate 1
```

La colonne **RAM** de l'onglet **Snapshots** passe alors à **Yes**. La restauration rendra la machine **en cours d'exécution**, processus compris, exactement dans l'état capturé.

7.4 Limites à connaître

- Les snapshots forment un **arbre** : **current** indique la position courante.
- Ils ne remplacent **pas** une sauvegarde : ils reposent sur le même stockage que la VM. Si ce stockage est perdu, les snapshots le sont aussi.
- Les accumuler dégrade les performances et consomme de l'espace ; les supprimer dès qu'ils ne servent plus.

8. Dépannage

« can't lock file '/var/lock/qemu-server/lock-.conf' - got timeout »

Proxmox protège chaque VM par un verrou pour qu'une seule opération modifie sa configuration à la fois. Le message signifie qu'une autre tâche le détenait — souvent une action lancée en parallèle depuis l'interface web.

```
qm config <vmid> | grep -i '^lock'    # une ligne "lock:" = verrou persistant
qm unlock <vmid>                      # uniquement si un verrou persiste
```

En l'absence de ligne **lock:**, le verrou était temporaire : il suffit de relancer la commande.

« swtpm_setup: Not overwriting existing state file. »

Message **informatif** du TPM logiciel, qui réutilise son état plutôt que d'en générer un neuf. Le démarrage a réussi.

La commande **Shutdown** ne fait rien

Comportement attendu si aucun système d'exploitation ne traite le signal ACPI. Utiliser **Stop**, en ayant conscience du caractère brutal de l'opération.

Collage impossible ou caractères **^[[200~** dans le terminal

Bracketed paste mal interprété. Correctif permanent :

```
echo 'set enable-bracketed-paste off' >> ~/.inputrc
```

Vérifier également que le *Console Viewer* est réglé sur **xterm.js** et non noVNC (**Datacenter** → **Options**).

La VM ne démarre pas en UEFI (« no bootable device », shell UEFI)

Secure Boot peut refuser un média non signé. Recréer le disque EFI sans clés pré-enregistrées :

```
qm set <vmid> --efidisk0 <storage>:1,efitype=4m,pre-enrolled-keys=0
```

9. Mémento des commandes

--- Environnement ---

pveversion	# version de Proxmox
pvesm status	# stockages et espace libre
qm list	# VM du nœud

--- Cycle de vie d'une VM ---

qm start <vmid>	# démarrer
qm shutdown <vmid>	# arrêt propre (ACPI)
qm stop <vmid>	# arrêt forcé
qm reset <vmid>	# redémarrage brutal
qm destroy <vmid>	# suppression définitive

--- Consultation ---

qm status <vmid>	# statut synthétique
qm status <vmid> --verbose grep ^[a-z]	# état d'exécution détaillé
qm config <vmid>	# configuration matérielle
cat /etc/pve/qemu-server/<vmid>.conf	# le même, fichier brut

--- Ressources ---

qm set <vmid> --memory 4096	# RAM en MiB
qm set <vmid> --cores 2	# cœurs par socket
qm set <vmid> --sockets 1	# sockets

```

qm resize <vmid> <disque> +10G      # agrandir un disque

# --- Snapshots ---

qm snapshot    <vmid> <nom> --description "...
qm snapshot    <vmid> <nom> --vmstate 1    # capture aussi la RAM
qm listsnapshot <vmid>
qm rollback    <vmid> <nom>
qm delsnapshot <vmid> <nom>

# --- Dépannage ---

qm unlock <vmid>      # lever un verrou persistant

# --- Conteneurs LXC (équivalents) ---

pct list / pct start / pct stop / pct enter <vmid>
pct exec <vmid> -- <commande>
pct config <vmid> / pct set <vmid> --memory 1024

```

Correspondance VM ↔ conteneur

Opération	VM (KVM)	Conteneur (LXC)
Lister	<code>qm list</code>	<code>pct list</code>
Démarrer	<code>qm start <id></code>	<code>pct start <id></code>
Accès shell	<code>qm terminal <id></code>	<code>pct enter <id></code>
Configurer	<code>qm set <id> --...</code>	<code>pct set <id> --...</code>
Sauvegarder	<code>vzdump <id></code>	<code>vzdump <id></code>

*Documentation rédigée et testée le 22/09/2026 sur Proxmox VE 9.2.2 (nœud **pve-haitem**).*